

Software Correctness - Project Report

Group members:

Nhat Thien Ngo

Daniel Djuve Krabbe

Mohammad Rajabi

The Architecture

The code in Java is responsible for the GUI. It reads the text in the command section and calls a Scala function that returns a data structure in the form of a Java record. The returned data structure contains all the x and y coordinates for each pixel, together with a color. The data structure also contains an error string. The parsed text from the TEXT-AT commandos are also a part of the data structure because the text is drawn directly in Java.

The Scala parts are responsible for the parsing and for error detection, and for the 3 drawing functions "drawLine()", "drawRectangle()", and "drawCircle()". In addition, the highlight on the last drawn commando is also handled in Scala. Because the text drawings are not handled in Scala but sent directly back to Java after parsing, the highlight functionality does not include the drawn text.

Here is the interface for the "drawCircle()" function. The "drawLine()" and "drawRectangle()" functions follow the same logic.

The argument 'command' contains information about whether it is a 'draw' or 'fill' command, and the color

```
def drawCircle(command: Command, p1: IntPoint, r: Int): Map[(Int, Int), Color] =...
```

The parser is made so that it also supports comments. An single line comment is ";" and a block can be commented with "(command ...)"

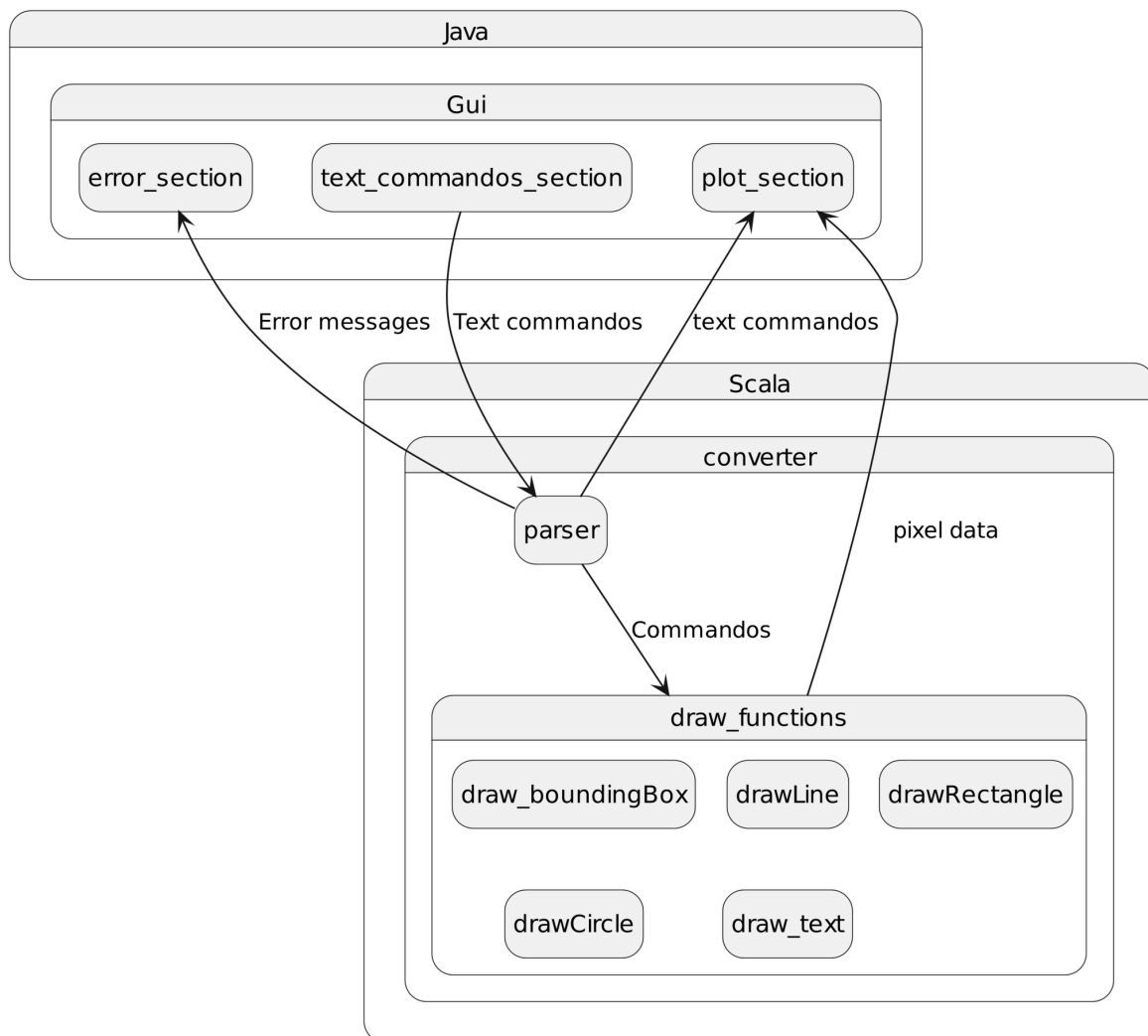
Command section code example:

```
(BOUNDING-BOX (0 0) (200 300))

; Figure: bar chart
(DRAW black
  ; 'Scala' bar commands
  (TEXT-AT (9 177.2) Scala)
  (TEXT-AT (61.6 177.2) 5)
  (fill red (RECTANGLE (30 174) (56.6 182)))
```

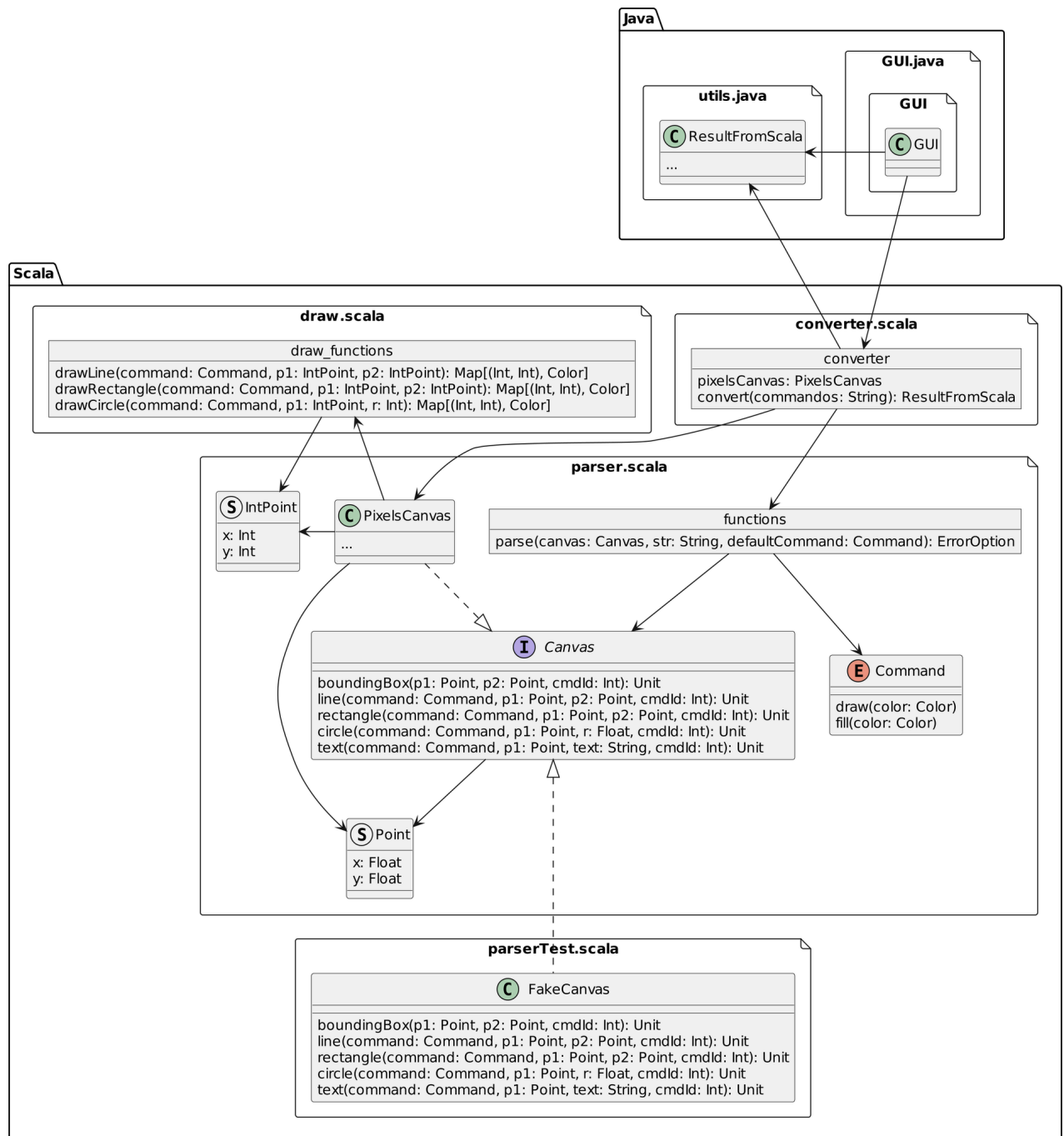
Conceptual Overview

A simplified overview of the data flow is shown in the following diagram.

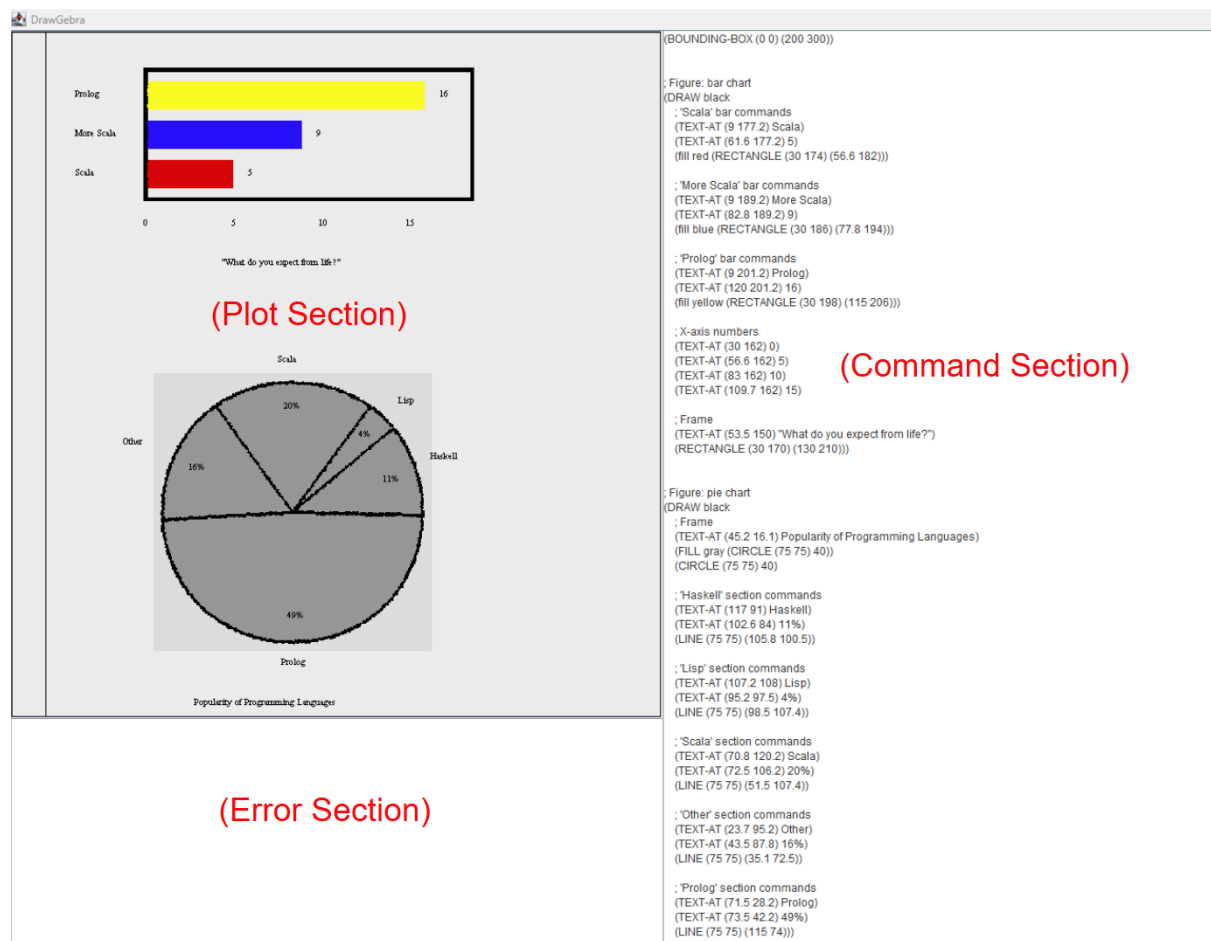


Class Diagram

Simplified class diagram of the dependencies



Application Interface and Usage Example



The application interface is divided into three functional modules. The Plot Section for visualization, the Error Section for error notification, and the Command Section for user input.

Note: In order to create smooth, precise drawings, we implemented the drawings with a 1:3 pixel scale ratio where the drawn lines are 3 pixels in width.

Testing and Proof Techniques

The GUI is tested by visually verifying that the functionality is as intended. The parser is tested by use of unit tests. Logika is used to verify inbound checks and to ensure loops can make progress.

Instead of handling the strings as sequences, as we have done in Slang, we can in Scala avoid out-of-bounds checks by using techniques that are built on iterators or string functions. Using an iterator-based for loop instead of a while loop also means we do not need to verify that the loop will terminate.

We implemented and compared two different approaches to verify the "drawRectangle()" function. Firstly, a simplified version of the "drawRectangle()" function has been implemented, containing a while loop with an invariant defined for the generated sequence. However, we have not managed to make Logika verify that the loop invariant holds. We do

not know if this is because Logika finds an edge case where the invariant does not hold or because Logika times out before concluding.

```
@pure def drawRectangle(x1: Z, y1: Z, x2: Z, y2: Z): MSZ[(Z, Z)] = {
  Contract(
    Requires(
      x1 <= x2, y1 <= y2
    ),
    Ensures(
      Res[MSZ[(Z, Z)]] == (x2 - x1 + 1) * (y2 - y1 + 1),
      Res[MSZ[(Z, Z)]] == drawRectangleRec(0, (x2 - x1 + 1) * (y2 - y1 + 1), x1, x2, y1, MSZ[(Z, Z)]())
    )
  )

  var pixels: MSZ[(Z, Z)] = MSZ[(Z, Z)]()
  var x: Z = x1
  var y: Z = y1
  val xMin = min(x1, x2)
  val xMax = max(x1, x2)
  val yMin = min(y1, y2)
  val yMax = max(y1, y2)
  Deduce(|- (xMin <= xMax))
  Deduce(|- (yMin <= yMax))

  while(y <= y2) {
    Invariant(
      Modifies(x, y, pixels),
      x >= x1,
      y >= y1,
      pixels.size == pixelsSize(x, y, x1, x2, y1),
      pixels == drawRectangleRec(
        0,
        pixelsSize(x, y, x1, x2, y1),
        x1,
        x2,
        y1,
        MSZ[(Z, Z)]()
      )
    )
    pixels = pixels :+ (x, y)
    if (x < x2) {
      x = x + 1
    } else {
      x = x1
      y = y + 1
    }
  }
  return pixels
}
```

We have made a recursive version that Logika can verify.

```
@strictpure def drawRectangleRecHelper(x: Z, y: Z, x1: Z, y1: Z, x2: Z, y2: Z): MSZ[(Z, Z)] = {
  Contract(
    Requires(x1 <= x2, y1 <= y2),
  )
  if(x == x2 & y == y2) {
    MSZ[(Z, Z)]((x, y))
  } else if(x < x2) {
    (x, y) :+ drawRectangleRecHelper(x+1, y, x1, y1, x2, y2)
  } else {
    (x, y) :+ drawRectangleRecHelper(x1, y+1, x1, y1, x2, y2)
  }
}

@strictpure def drawRectangleRec(x1: Z, y1: Z, x2: Z, y2: Z): MSZ[(Z, Z)] = {
  val xMin = min(x1, x2)
  val xMax = max(x1, x2)
  val yMin = min(y1, y2)
  val yMax = max(y1, y2)
  Deduce(|- (xMin <= xMax))
  Deduce(|- (yMin <= yMax))
  drawRectangleRecHelper(xMin, yMin, xMin, yMin, xMax, yMax)
}
```

The following code demonstrates how the "text.isInBound()" is used to verify that the first index is safe to access.

```
@strictpure def head(text: MSZ[String]): String = {  
  if (text.isInBound(0)) {  
    text(0)  
  } else {  
    ""  
  }  
}
```

Moreover, this simple "Head()" function is then later on used to define more advanced functions where the inbound check is not needed, as we have done in the following code.

```
@strictpure def removeCommentsHelper(first: String, rest: MSZ[String], commentActive: B): MSZ[String] = {  
  if (first == "") {  
    MSZ[String]()  
  } else if (first == "\n") {  
    removeCommentsHelper(head(rest), tail(rest), false)  
  } else if (first == ";" | commentActive) {  
    removeCommentsHelper(head(rest), tail(rest), true)  
  } else {  
    first += removeCommentsHelper(head(rest), tail(rest), false)  
  }  
}  
  
@strictpure def removeComments(inData: MSZ[String]): MSZ[String] = {  
  removeCommentsHelper(head(inData), tail(inData), false)  
}
```

Limitations and areas of improvement

1. The current GUI lacks zoom constraints. Without established minimum and maximum scale limits, users can zoom in/out indefinitely, leading to potential rendering issues or loss of visual context.
2. The current interface immediately processes all command inputs, which limits user control. Introducing dedicated 'Run' and 'Clear' functions would improve the user experience by allowing for manual execution and easier workspace management.
3. Current error reporting lacks precision and detail. Integrating line-specific details would significantly improve diagnostic clarity.